

Lab 2. Accessing Table Data, DML Statements, and Transactions

Write SQL queries to:

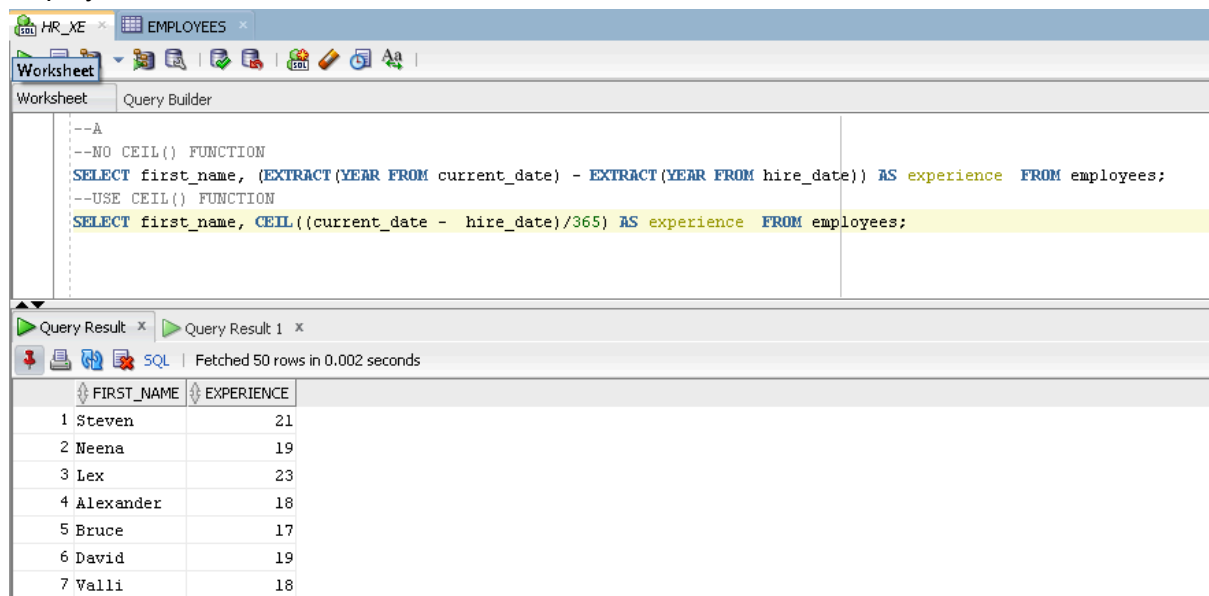
a. Display first name and experience of the employees.

--NO CEIL() FUNCTION

```
SELECT first_name, (EXTRACT(YEAR FROM current_date) - EXTRACT(YEAR FROM hire_date)) AS experience FROM employees;
```

--USE CEIL() FUNCTION

```
SELECT first_name, CEIL((current_date - hire_date)/365) AS experience FROM employees;
```

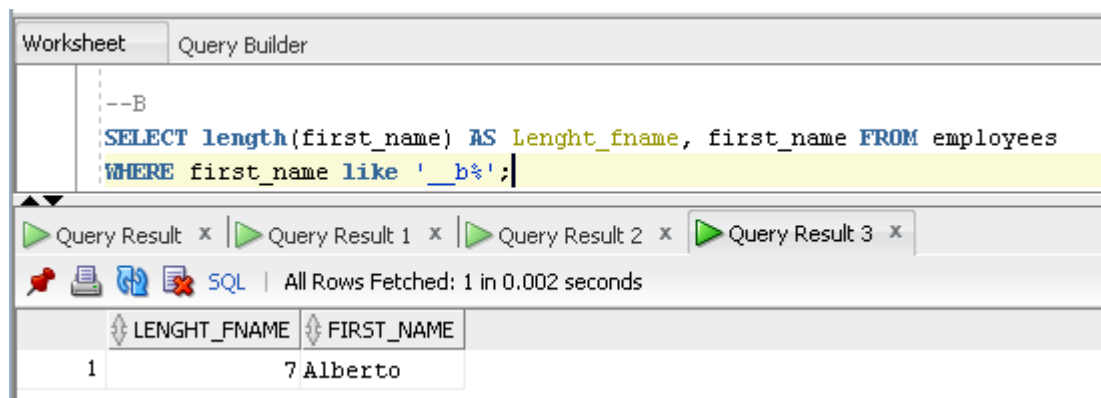


The screenshot shows the SQL Developer interface. The query editor contains two SQL queries. The first query calculates experience by subtracting the year from the hire date from the current year. The second query uses the CEIL function to calculate experience based on the number of days since hire date divided by 365. The query results pane shows the results of the second query, displaying the first name and experience for seven employees.

FIRST_NAME	EXPERIENCE
1 Steven	21
2 Neena	19
3 Lex	23
4 Alexander	18
5 Bruce	17
6 David	19
7 Valli	18

b. Display the length of first name for employees where last name contain character 'b' after 3rd position.

```
SELECT length(first_name) AS Lenght_fname, first_name FROM employees  
WHERE first_name like '__b%';
```



The screenshot shows the SQL Developer interface. The query editor contains a SQL query that selects the length of the first name and the first name for employees whose first name contains the character 'b' at the third position. The query results pane shows the results of this query, displaying the length of the first name and the first name for one employee, Alberto.

LENGHT_FNAME	FIRST_NAME
1	7 Alberto

c. Display employees who joined in the current year.

SELECT * FROM employees

Where EXTRACT(YEAR FROM hire_date) = EXTRACT(YEAR FROM current_date);

The screenshot shows the SQL Developer interface with the Query Builder tab active. The query editor contains the following SQL statement:

```
--C
SELECT * FROM employees
Where EXTRACT(YEAR FROM hire_date) = EXTRACT(YEAR FROM current_date);
```

Below the query editor, the Query Result tab is visible, showing the column list for the result set:

EMPLOYEE...	FIRST_NA...	LAST_NAME	EMAIL	PHONE_N...	HIRE_DATE	JOB_ID
-------------	-------------	-----------	-------	------------	-----------	--------

d. Display job ID for jobs with average salary more than 10,000.

SELECT Job_id FROM jobs

where (min_salary + max_salary)/2 > 10000;

The screenshot shows the SQL Developer interface with the Query Builder tab active. The query editor contains the following SQL statement:

```
--D
SELECT Job_id FROM jobs
where (min_salary + max_salary)/2 > 10000;
```

Below the query editor, the Query Result tab is visible, showing the column list for the result set:

JOB_ID
1 AD_PRE
2 AD_V
3 FI_MGR
4 AC_MGR
5 SA_MAN
6 PU_MAN
7 MK_MAN

e. Display the first name, last name, salary and job grade for all employees.

SELECT e.first_name, e.last_name, e.salary, j.job_title

FROM employees e, jobs j

WHERE e.job_id = j.job_id;

Worksheet

Query Builder

--E

SELECT e.first_name, e.last_name, e.salary, j.job_title

FROM employees e, jobs j

WHERE e.job_id = j.job_id;

Query Result x

 SQL | Fetched 50 rows in 0.003 seconds

	FIRST_NAME	LAST_NAME	SALARY	JOB_TITLE
1	William	Gietz	8300	Public Accountant
2	Shelley	Higgins	12008	Accounting Manager
3	Jennifer	Whalen	4400	Administration Assistant
4	Steven	King	24000	President
5	Lex	De Haan	17000	Administration Vice President
6	Neena	Kochhar	17000	Administration Vice President
7	John	Chen	8200	Accountant
8	Daniel	Faviet	9000	Accountant
9	Luis	Popp	6900	Accountant
10	Ismael	Sciarra	7700	Accountant
11	Jose Manuel	Urman	7800	Accountant
12	Nancy	Greenberg	12008	Finance Manager

f. Display the first name, last name, department number and department name for all employees for departments 80 or 40.

```
SELECT e.first_name, e.last_name, d.department_id, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE d.department_id IN (80, 40);
```

WorksheetQuery Builder

--F

SELECT e.first_name, e.last_name, d.department_id, d.department_name

FROM employees e

JOIN departments d ON e.department_id = d.department_id

WHERE d.department_id IN (80, 40);

Query Result x

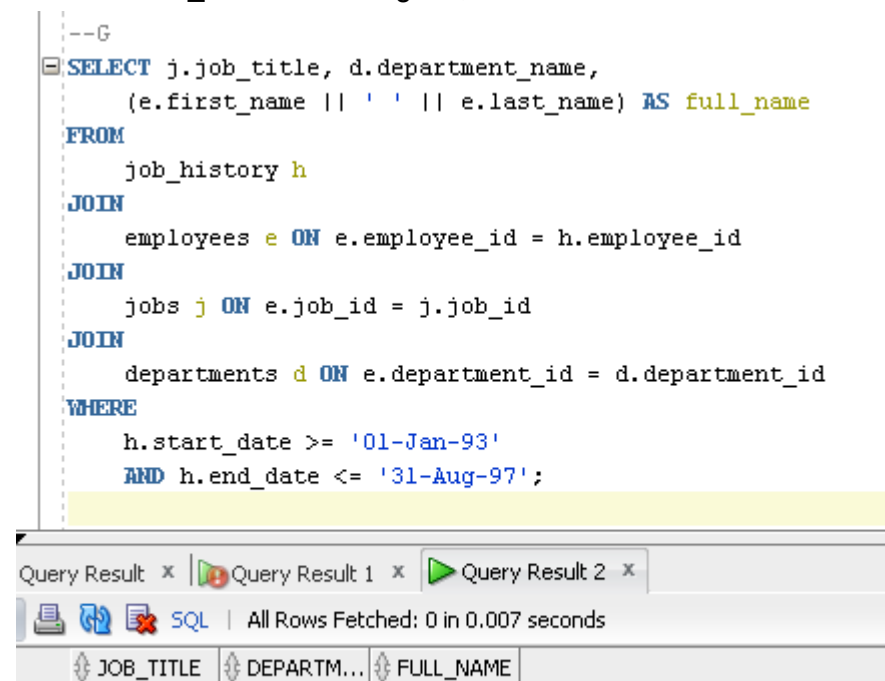
SQL

All Rows Fetched: 35 in 0.002 seconds

	FIRST_NAME	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Susan	Mavris	40	Human Resources
2	John	Russell	80	Sales
3	Karen	Partners	80	Sales
4	Alberto	Errazuriz	80	Sales
5	Gerald	Cambrault	80	Sales
6	Eleni	Zlotkey	80	Sales
7	Peter	Tucker	80	Sales

g. Display the job title, department name, full name (first and last name) of employee and starting date for all the jobs which started on or after 1st January, 1993 and ending with on or before 31 August, 1997.

```
SELECT j.job_title, d.department_name,  
       (e.first_name || ' ' || e.last_name) AS full_name  
FROM  
  job_history h  
JOIN  
  employees e ON e.employee_id = h.employee_id  
JOIN  
  jobs j ON e.job_id = j.job_id  
JOIN  
  departments d ON e.department_id = d.department_id  
WHERE  
  h.start_date >= '01-Jan-93'  
  AND h.end_date <= '31-Aug-97';
```



h. Display the department name and number of employees in each of the department.

```
SELECT d.department_name, count(*) AS num_employees  
FROM employees e  
JOIN departments d ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name;
```

```
--H
SELECT d.department_name, count(*) AS num_employees
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;
```

DEPARTMENT_NAME	NUM_EMPLOYEES
1 Finance	6
2 Shipping	45
3 Public Relations	1
4 Purchasing	6
5 Executive	3
6 Administration	1
7 Accounting	2
8 Human Resources	1
9 Marketing	2
10 IT	5
11 Sales	34

i. Display years in which more than 10 employees joined.

```
SELECT EXTRACT(YEAR FROM hire_date)-- count(*)
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
HAVING count(*)>10;
```

EXTRACT(YEARFROMHIRE_DATE)	COUNT(*)
1 2005	29
2 2006	24
3 2007	19
4 2008	11

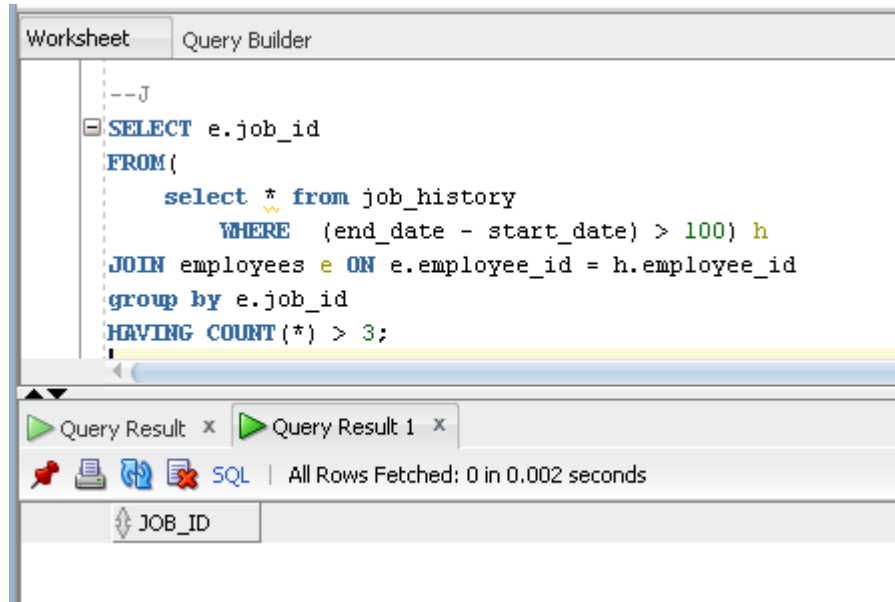
j. Display job ID of jobs that were done by more than 3 employees for more than 100 days.

```
SELECT e.job_id
FROM(
```

```

select * from job_history
WHERE (end_date - start_date) > 100) h
JOIN employees e ON e.employee_id = h.employee_id
group by e.job_id
HAVING COUNT(*) > 3;

```



k. Change job ID of the employee 110 to IT_PROG if the employee belongs to department 10 and the existing job ID does not start with IT.

```

UPDATE employees
SET job_id = 'IT_PROG'
WHERE employee_id = 110
AND department_id = 10
AND job_id NOT LIKE 'IT%';

```

